

Python Programmation

CM 1 : Welcome and Introduction

Julien Bezançon

julien.bezancon@univ-lorraine.fr / <https://julienbez.github.io>



1 Before we start...

2 Introduction

3 The Basics of Python

4 Control Flow in Python

5 Objects and Types

About me



You will see me in...

- Data Storage and Retrieval M1
- Prompt Engineering M2
- Software Projects M2
- Written Corpora M2
- Lexicology M2

About this course

8 lectures (CM), each one followed by a practical session (TD)

- General Introduction
- Data Structure
- Functions
- Object-Oriented Python
- Functional Programming

About this course

8 lectures (CM), each one followed by a practical session (TD)

- General Introduction
- Data Structure
- Functions
- Object-Oriented Python
- Functional Programming

Evaluation mode

- Practical sessions (50 %)
- Final exam (50 %)

About this course

8 lectures (CM), each one followed by a practical session (TD)

- General Introduction
- Data Structure
- Functions
- Object-Oriented Python
- Functional Programming

Evaluation mode

- Practical sessions (50 %)
- Final exam (50 %)

Final exam : 30/11/2026

About the practical sessions

For each practical session...

- Download the notebook on Arche
- Complete it and put it back on Arche
- 3 random practical sessions will be graded
- the worst grade *can* be removed

References for this course

- Tom Bourgeade (tom.bourgeade@loria.fr)
- Sam Redmon (sredmond@stanford.edu)
- Python documentation¹

¹<https://docs.python.org/3/>

① Before we start...

② Introduction

③ The Basics of Python

④ Control Flow in Python

⑤ Objects and Types

What is Python ?

Python is a programming language

- Who ? Guido van Rossum, a.k.a the *Benevolent Dictator for Life*
- Why ? Because he wanted a hobby during christmas
- The name ? A reference to *Monty Python's Flying Circus*

What is Python ?

Python is a programming language

- Who ? Guido van Rossum, a.k.a the *Benevolent Dictator for Life*
- Why ? Because he wanted a hobby during christmas
- The name ? A reference to *Monty Python's Flying Circus*

Python is an ongoing project

- Python 1 : 1994
- Python 2 : 2000
- Python 3 : 2008
- As of today : Python 3.14.7

What is Python ?

Python is a programming language

- Who ? Guido van Rossum, a.k.a the *Benevolent Dictator for Life*
- Why ? Because he wanted a hobby during christmas
- The name ? A reference to *Monty Python's Flying Circus*

Python is an ongoing project

- Python 1 : 1994
- Python 2 : 2000
- Python 3 : 2008
- As of today : Python 3.14.7

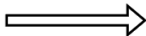
Why should I use Python ?

First reason : It's easy

The "Hello World" script

Script Window

```
Function Main  
    Output "Hello world!"  
End
```



Command line

```
> Hello world
```

First reason : It's easy

"Hello World" in Java

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

First reason : It's easy

"Hello World" in C++

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World";
    return 0;
}
```

First reason : It's easy

"Hello World" in Python

```
print("Hello World")
```

Second reason : Python is very interactive

Python can even be used in the command line !

```
C:/Users/julien/documents/github > python
```

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) on win32  
Type "help", "copyright", "credits" or "license" for more information.
```

```
>>>
```

Second reason : Python is very interactive

Python can even be used in the command line !

```
C:/Users/julien/documents/github > python
```

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) on win32  
Type "help", "copyright", "credits" or "license" for more information.
```

```
>>> print("Hello World")  
Hello World
```

Second reason : Python is very interactive

Why does it matter ?

- Immediate gratification
- Lots of freely available resources to learn Python
- Shortens code-test-debug cycle to seconds

Second reason : Python is very interactive

Why does it matter ?

- Immediate gratification
- Lots of freely available resources to learn Python
- Shortens code-test-debug cycle to seconds

Python is also...

- Easy to install (Linux, Windows, IOS,etc)
- Modular (you can import libraries)
- + you can contribute to Python !

Third reason : Python is widely used

Who uses Python ?

- Researchers from various fields (NLP, astronomy, biology, social sciences, etc)

Third reason : Python is widely used

Who uses Python ?

- Researchers from various fields (NLP, astronomy, biology, social sciences, etc)
- Manufacturers and companies of all size

Third reason : Python is widely used

Who uses Python ?

- Researchers from various fields (NLP, astronomy, biology, social sciences, etc)
- Manufacturers and companies of all size

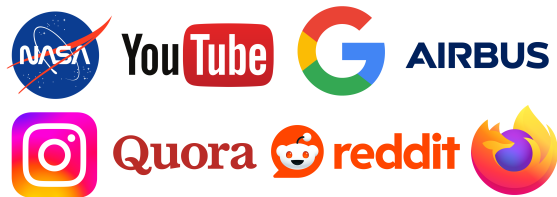


Figure 1: Some examples of companies using Python

Third reason : Python is widely used

More specifically, Python is used for...

- Data science, Statistical analysis
- Software development
- Cybersecurity
- Financial technologies
- Artificial Intelligence
- Web development
- Machine Learning systems

And the list goes on...

A Quick summary

Python is...

- Popular (million of users worldwide)
- Practical (easy to install, easy to use)
- Still evolving²

²<https://devguide.python.org/versions/>

A Quick summary

Python is...

- Popular (million of users worldwide)
- Practical (easy to install, easy to use)
- Still evolving²

During this course, you will...

- Develop skills with Python fundamentals
- Learn to recognize "bad" Python and to write "good" Python
- Frequently practice python, both during lectures and practical sessions

²<https://devguide.python.org/versions/>

Install Python: <https://www.python.org/downloads/>

Do you have a Python editor on your computer ?
If not, then install one (e.g: Jupyter Notebook³)

³<https://jupyter.org/install>

① Before we start...

② Introduction

③ The Basics of Python

④ Control Flow in Python

⑤ Objects and Types

Variables

```
x = 42  
sentence = "Hello"
```

Questions

- What is the difference between these two variables ?
- Is there a way to show this difference in Python ?
- What would be the result of these operations ?

Variables

```
x = 42
sentence = "Hello"

x * 2
sentence + "World"
```

Variables

```
x = 42
sentence = "Hello"

x * 2
sentence + "World"
```

Questions

- What is the difference between these two variables ?
- Is there a way to show this difference in Python ?
- What would be the result of these operations ?

Variables

```
x = 42
sentence = "Hello"

x * 2
sentence + "World"
```

Questions

- What is the difference between these two variables ? $\Rightarrow$ their type
- Is there a way to show this difference in Python ?
- What would be the result of these operations ?

Variables

```
x = 42
sentence = "Hello"

x * 2
sentence + "World"
```

Questions

- What is the difference between these two variables ? $\Rightarrow$ their type
- Is there a way to show this difference in Python ? $\Rightarrow$ `type()`
- What would be the result of these operations ?

Variables

```
x = 42
sentence = "Hello"

x * 2
sentence + "World"
```

Questions

- What is the difference between these two variables ? $\Rightarrow$ their type
- Is there a way to show this difference in Python ? $\Rightarrow$ `type()`
- What would be the result of these operations ? $\Rightarrow$ 84 and "Hello World"

Types

```
x = 42
sentence = "Hello"

type(x)
type(sentence)
```

Types

```
x = 42
sentence = "Hello"

type(x)
type(sentence)
```

Questions

- What is the type of **x** ?
- What is the type of **sentence** ?

Types

```
x = 42
sentence = "Hello"

type(x)
type(sentence)
```

Questions

- What is the type of **x**? $\Rightarrow$ integer (int)
- What is the type of **sentence**?

Types

```
x = 42
sentence = "Hello"

type(x)
type(sentence)
```

Questions

- What is the type of **x**? $\Rightarrow$ integer (int)
- What is the type of **sentence**? $\Rightarrow$ string (str)

Types

```
x = 42
sentence = "Hello"

type(x)
type(sentence)
```

Questions

- What is the type of **x**? $\Rightarrow$ integer (int)
- What is the type of **sentence**? $\Rightarrow$ string (str)

Variables in Python

- Dynamically-typed: declared without an explicit type
- Python knows the type of a variable when it is created

Comments

```
x = 42
sentence = "Hello"

x * 2
sentence + "World"

type(x)
type(sentence)
```

Comments

```
x = 42                                # A variable
sentence = "Hello"

x * 2
sentence + "World"

type(x)
type(sentence)
```

Comments

```
x = 42                                # A variable
sentence = "Hello"                   # Another variable

x * 2
sentence + "World"

type(x)
type(sentence)
```

Comments

```
x = 42                                # A variable
sentence = "Hello"                   # Another variable

x * 2                                # A multiplication
sentence + "World"

type(x)
type(sentence)
```

Comments

```
x = 42                                # A variable
sentence = "Hello"                   # Another variable

x * 2                                # A multiplication
sentence + "World"                   # => "Hello World"

type(x)
type(sentence)
```

Comments

```
x = 42                                # A variable
sentence = "Hello"                   # Another variable

x * 2                                # A multiplication
sentence + "World"                   # => "Hello World"

type(x)                              # => <class 'int'>
type(sentence)
```

Comments

```
x = 42                                # A variable
sentence = "Hello"                   # Another variable

x * 2                                # A multiplication
sentence + "World"                   # => "Hello World"

type(x)                              # => <class 'int'>
type(sentence)                      # => <class 'str'>
```

Comments

```
x = 42                # A variable
sentence = "Hello"    # Another variable

x * 2                 # A multiplication
sentence + "World"    # => "Hello World"

type(x)               # => <class 'int'>
type(sentence)        # => <class 'str'>
```

```
"""
You can also write multiline comments
between three quotation marks.
"""
```

Numbers and Arithmetic Operators

Two numeric types : integer and float

```
3
```

```
3.0
```

Numbers and Arithmetic Operators

Two numeric types : integer and float

```
3                # => Integer (int)
3.0
```

Numbers and Arithmetic Operators

Two numeric types : integer and float

```
3                # => Integer (int)
3.0              # => Float (float)
```

Numbers and Arithmetic Operators

Two numeric types : integer and float

```
3                # => Integer (int)
3.0             # => Float (float)
```

Arithmetic operators

```
1 + 1
2 - 1
10 * 2
5 / 2
7 // 3
7 % 3
2 ** 4
```

Numbers and Arithmetic Operators

Two numeric types : integer and float

```
3                # => Integer (int)
3.0             # => Float (float)
```

Arithmetic operators

```
1 + 1            # => 2
2 - 1
10 * 2
5 / 2
7 // 3
7 % 3
2 ** 4
```

Numbers and Arithmetic Operators

Two numeric types : integer and float

```
3                # => Integer (int)
3.0             # => Float (float)
```

Arithmetic operators

```
1 + 1           # => 2
2 - 1           # => 1
10 * 2
5 / 2
7 // 3
7 % 3
2 ** 4
```

Numbers and Arithmetic Operators

Two numeric types : integer and float

```
3                # => Integer (int)
3.0             # => Float (float)
```

Arithmetic operators

```
1 + 1            # => 2
2 - 1            # => 1
10 * 2           # => 20
5 / 2
7 // 3
7 % 3
2 ** 4
```

Numbers and Arithmetic Operators

Two numeric types : integer and float

```
3                # => Integer (int)
3.0              # => Float (float)
```

Arithmetic operators

```
1 + 1            # => 2
2 - 1            # => 1
10 * 2           # => 20
5 / 2            # => 2.5
7 // 3
7 % 3
2 ** 4
```

Numbers and Arithmetic Operators

Two numeric types : integer and float

```
3                # => Integer (int)
3.0              # => Float (float)
```

Arithmetic operators

```
1 + 1            # => 2
2 - 1            # => 1
10 * 2           # => 20
5 / 2            # => 2.5
7 // 3           # => 2 (integer division)
7 % 3
2 ** 4
```

Numbers and Arithmetic Operators

Two numeric types : integer and float

```
3                # => Integer (int)
3.0             # => Float (float)
```

Arithmetic operators

```
1 + 1           # => 2
2 - 1           # => 1
10 * 2          # => 20
5 / 2           # => 2.5
7 // 3          # => 2 (integer division)
7 % 3           # => 1 (integer modulus)
2 ** 4
```

Numbers and Arithmetic Operators

Two numeric types : integer and float

```
3                # => Integer (int)
3.0             # => Float (float)
```

Arithmetic operators

```
1 + 1            # => 2
2 - 1            # => 1
10 * 2           # => 20
5 / 2            # => 2.5
7 // 3           # => 2 (integer division)
7 % 3            # => 1 (integer modulus)
2 ** 4           # => 16 (exponentiation)
```

Booleans and Comparison Operators

A boolean (bool) is a subtype of integer (either 1 or 0)

```
True
False
not True
True and False
True or False
```

Booleans and Comparison Operators

A boolean (bool) is a subtype of integer (either 1 or 0)

```
True                # => True (1)
False
not True
True and False
True or False
```

Booleans and Comparison Operators

A boolean (bool) is a subtype of integer (either 1 or 0)

```
True                # => True (1)
False               # => False (0)
not True
True and False
True or False
```

Booleans and Comparison Operators

A boolean (bool) is a subtype of integer (either 1 or 0)

```
True                # => True (1)
False               # => False (0)
not True            # => False
True and False
True or False
```

Booleans and Comparison Operators

A boolean (bool) is a subtype of integer (either 1 or 0)

```
True                # => True (1)
False               # => False (0)
not True            # => False
True and False      # => False
True or False
```

Booleans and Comparison Operators

A boolean (bool) is a subtype of integer (either 1 or 0)

```
True                # => True (1)
False               # => False (0)
not True            # => False
True and False      # => False
True or False       # => True
```

Booleans and Comparison Operators

A boolean (bool) is a subtype of integer (either 1 or 0)

```
True                # => True (1)
False               # => False (0)
not True            # => False
True and False      # => False
True or False       # => True
```

Comparison operators

```
1 == 1
1 != 1
2 * 3 != 5
1 < 10
2 >= 0
```

Booleans and Comparison Operators

A boolean (bool) is a subtype of integer (either 1 or 0)

```
True                # => True (1)
False               # => False (0)
not True            # => False
True and False     # => False
True or False      # => True
```

Comparison operators

```
1 == 1              # => True (equal to)
1 != 1
2 * 3 != 5
1 < 10
2 >= 0
```

Booleans and Comparison Operators

A boolean (bool) is a subtype of integer (either 1 or 0)

```
True                # => True (1)
False               # => False (0)
not True            # => False
True and False     # => False
True or False      # => True
```

Comparison operators

```
1 == 1              # => True (equal to)
1 != 1              # => False (not equal to)
2 * 3 != 5
1 < 10
2 >= 0
```

Booleans and Comparison Operators

A boolean (bool) is a subtype of integer (either 1 or 0)

```
True                # => True (1)
False               # => False (0)
not True            # => False
True and False     # => False
True or False      # => True
```

Comparison operators

```
1 == 1              # => True (equal to)
1 != 1              # => False (not equal to)
2 * 3 != 5          # => True
1 < 10
2 >= 0
```

Booleans and Comparison Operators

A boolean (bool) is a subtype of integer (either 1 or 0)

```
True                # => True (1)
False               # => False (0)
not True            # => False
True and False     # => False
True or False      # => True
```

Comparison operators

```
1 == 1              # => True (equal to)
1 != 1              # => False (not equal to)
2 * 3 != 5          # => True
1 < 10              # => True (greater than)
2 >= 0
```

Booleans and Comparison Operators

A boolean (bool) is a subtype of integer (either 1 or 0)

```
True                # => True (1)
False               # => False (0)
not True            # => False
True and False     # => False
True or False      # => True
```

Comparison operators

```
1 == 1              # => True (equal to)
1 != 1              # => False (not equal to)
2 * 3 != 5          # => True
1 < 10              # => True (greater than)
2 >= 0              # => True (less than or equal to)
```

Strings

A string is always contained between quotation marks

```
a_string = 'Hello'           # The quotation marks can be an apostrophe
b_string = "Wørld"           # Unicode by default
```

Strings

A string is always contained between quotation marks

```
a_string = 'Hello'           # The quotation marks can be an apostrophe
b_string = "Wørld"           # Unicode by default

final_string = a_string + ' ' + b_string + "!"
# => "Hello Wørld!"
```

String Indexes

Each character of a string has its own index

```
s = "Julien"
```

String Indexes

Each character of a string has its own index

```
s = "Julien"
```

```
# Index start at 0
```

```
s[0] == "J"
```

```
s[1] == "u"
```

```
...
```

```
s[5] == "n"
```

String Indexes

Each character of a string has its own index

```
s = "Julien"

# Index start at 0
s[0] == "J"
s[1] == "u"
...
s[5] == "n"

# Also work with negative indexes
s[-6] == "J"
s[-5] == "u"
...
s[-1] == "n"
```

String Slicing

Slicing is used to fetch a subpart of a string

```
s = "Julien"
```

String Slicing

Slicing is used to fetch a subpart of a string

```
s = "Julien"

# Fetch characters between two indexes
s[0:2] == "Ju"
s[3:6] == "ien"
s[1:4] == "uli"
```

String Slicing

Slicing is used to fetch a subpart of a string

```
s = "Julien"

# Fetch characters between two indexes
s[0:2] == "Ju"
s[3:6] == "ien"
s[1:4] == "uli"

s[:3] == "Jul"           # Implicitly starts at 0
s[3:] == "ien"          # Implicitly ends at the end
```

String Slicing

Slicing is used to fetch a subpart of a string

```
s = "Julien"

# Fetch characters between two indexes
s[0:2] == "Ju"
s[3:6] == "ien"
s[1:4] == "uli"

s[:3] == "Jul"           # Implicitly starts at 0
s[3:0] == "ien"          # Implicitly ends at the end

s[::-1] == "neiluj"      # One way to reverse a string
```

Converting Values

The type of an object can be changed

```
str(42)           # => "42" (int to string)
int("42")        # => 42 (string to int)
float("2.5")     # => 2.5 (string to float)
float("1")       # => 1.0
```

Converting Values

The type of an object can be changed

```
str(42)           # => "42" (int to string)
int("42")         # => 42 (string to int)
float("2.5")       # => 2.5 (string to float)
float("1")         # => 1.0
```

Questions

- In which case(s) converting a string to an integer is useful ?
- In which case(s) converting an integer to a string is useful ?

Lists

Lists are object which contain other objects

```
# Brackets delimit a list
# Commas separate its elements

a_list = [1,2,3]
b_list = ['a','b','c','d']
```

Lists

Lists are object which contain other objects

```
# Brackets delimit a list
# Commas separate its elements

a_list = [1,2,3]
b_list = ['a','b','c','d']

# A list can contain elements of different types
mixed_list = [4,5,"seconds"]
```

Lists

Lists are object which contain other objects

```
# Brackets delimit a list
# Commas separate its elements

a_list = [1,2,3]
b_list = ['a','b','c','d']

# A list can contain elements of different types
mixed_list = [4,5,"seconds"]

# Elements can be added to the end of a list
empty_list = []                # List without any element
empty_list.append(7)           # => empty_list == [7]
empty_list.append("kebab")     # => empty_list == [7,"kebab"]
```

List Indexing and Slicing

Same as string indexing and slicing

```
letters = ['a', 'b', 'c', 'd']  
numbers = [2, 3, 5, 7, 11]
```

List Indexing and Slicing

Same as string indexing and slicing

```
letters = ['a', 'b', 'c', 'd']
numbers = [2, 3, 5, 7, 11]

# Access elements at a particular index
numbers[0]           # => 2
numbers[-1]          # => 11
```

List Indexing and Slicing

Same as string indexing and slicing

```
letters = ['a', 'b', 'c', 'd']
numbers = [2, 3, 5, 7, 11]

# Access elements at a particular index
numbers[0]           # => 2
numbers[-1]          # => 11

# The same rules apply of slicing
letters[:3]           # => ['a', 'b', 'c']
numbers[1:-1]         # => [3, 5, 7]
```

Nested lists

Lists can even contain other lists !

```
letters = ['a', 'b', 'c', 'd']  
numbers = [2, 3, 5, 7, 11]
```

Nested lists

Lists can even contain other lists !

```
letters = ['a', 'b', 'c', 'd']
numbers = [2, 3, 5, 7, 11]

combo = [letters, numbers]    # => [['a', 'b', 'c', 'd'], [2, 3, 5, 7, 11]]
combo[0]                      # => ['a', 'b', 'c', 'd']
combo[0][1]                   # => 'b'
combo[1][2:]                  # => [5, 7, 11]
```

split Function

It is possible to convert a string to a list

```
# "split" partitions a string by a delimiter  
"ham cheese bacon".split()  
# => ["ham", "cheese", "bacon"]
```

split Function

It is possible to convert a string to a list

```
# "split" partitions a string by a delimiter
"ham cheese bacon".split()
# => ["ham", "cheese", "bacon"]

"03-30-2016".split(sep="-")
# => ["03", "30", "2016"]
```

split Function

It is possible to convert a string to a list

```
# "split" partitions a string by a delimiter
"ham cheese bacon".split()
# => ["ham", "cheese", "bacon"]

"03-30-2016".split(sep="-")
# => ["03", "30", "2016"]

# "join" creates a string from a list (of strings)
", ".join(["Eric", "John", "Michael"])
# => "Eric, John, Michael"
```

General Queries

some useful queries in Python

```
# Length (len)
len([])           # => 0
len("python")     # => 6
len([4, 5, "seconds"]) # => 3
```

General Queries

some useful queries in Python

Length (len)

```
len([]) # => 0
```

```
len("python") # => 6
```

```
len([4, 5, "seconds"]) # => 3
```

Membership (in)

```
0 in [] # => False
```

```
'y' in "python" # => True
```

```
"sec" in [4, 5, "seconds"] # => False
```

Console Input and Output

Read a string from the user with Python

```
name = input("What is your name? ")  
>>> What is your name ? Julien
```

Console Input and Output

Read a string from the user with Python

```
name = input("What is your name? ")
>>> What is your name ?  Julien

print("I'm Python.  Nice to meet you,", name)
# => I'm Python.  Nice to meet you, Julien
```

File Reading

```
# Function to open a file in read mode
with open("filepath.txt","r") as f:
    content = f.read()           # Save the content of the file in a string
```

File Reading

- File content $\Rightarrow$ Python data
- read all at once or line by line
- mode = "r" (for "read")

File Reading

```
# Function to open a file in read mode
with open("filepath.txt","r") as f:
    content = f.read()           # Save the content of the file in a string

with open("filepath.txt","r") as f:
    lines = f.readline(5)       # Save the first 5 lines in a list
```

File Reading

- File content $\Rightarrow$ Python data
- read all at once or line by line
- mode = "r" (for "read")

File Reading

```
# Function to open a file in read mode
with open("filepath.txt","r") as f:
    content = f.read()           # Save the content of the file in a string

with open("filepath.txt","r") as f:
    lines = f.readline(5)       # Save the first 5 lines in a list

with open("filepath.txt","r") as f:
    lines = f.readlines()       # Save each line of the file in a list
```

File Reading

```
# Function to open a file in read mode
with open("filepath.txt","r") as f:
    content = f.read()           # Save the content of the file in a string

with open("filepath.txt","r") as f:
    lines = f.readline(5)       # Save the first 5 lines in a list

with open("filepath.txt","r") as f:
    lines = f.readlines()       # Save each line of the file in a list
```

File Reading

- File content $\Rightarrow$ Python data
- read all at once or line by line
- mode = "r" (for "read")

File Writing

```
# Function to open a file in write mode
with open("filepath.txt","w") as f:
    f.write(a_string)           # Save a string to a file
```

File Writing

- Python data $\Rightarrow$ File content
- write all at once or line by line
- mode = "w" (for "write")

File Writing

```
# Function to open a file in write mode
with open("filepath.txt","w") as f:
    f.write(a_string)           # Save a string to a file

with open("filepath.txt","w") as f:
    f.writelines(a_list)       # save each element of a list in a line
```

File Writing

```
# Function to open a file in write mode
with open("filepath.txt","w") as f:
    f.write(a_string)           # Save a string to a file

with open("filepath.txt","w") as f:
    f.writelines(a_list)       # save each element of a list in a line
```

File Writing

- Python data $\Rightarrow$ File content
- write all at once or line by line
- mode = "w" (for "write")

Exercise Time !

1. Open the command line and type "Python"
2. Use the "input" function to ask the user (you) for a number
3. Multiply this number by 10 and print the result
4. What is the issue ? fix it
5. Check if the result is superior or equal to 10
6. Create a list and add the result to it

Don't forget to use variables !

- 1 Before we start...
- 2 Introduction
- 3 The Basics of Python
- 4 Control Flow in Python**
- 5 Objects and Types

if, elif and else Statements

a "if" statement evaluate if a condition is True or False

```
if the_world_is_flat:           # the_world_is_flat is a variable !  
    print("don't fall off!")
```

if, elif and else Statements

a "if" statement evaluate if a condition is True or False

```
if the_world_is_flat:           # the_world_is_flat is a variable !
    print("don't fall off!")

# Check several conditions
if some_condition:
    print("Some condition holds")
```

if, elif and else Statements

a "if" statement evaluate if a condition is True or False

```
if the_world_is_flat:           # the_world_is_flat is a variable !
    print("don't fall off!")

# Check several conditions
if some_condition:
    print("Some condition holds")

elif other_condition:           # Zero or more elif
    print("Other condition holds")
```

if, elif and else Statements

a "if" statement evaluate if a condition is True or False

```
if the_world_is_flat:           # the_world_is_flat is a variable !
    print("don't fall off!")

# Check several conditions
if some_condition:
    print("Some condition holds")

elif other_condition:           # Zero or more elif
    print("Other condition holds")

else:                           # else is optional
    print("Neither condition holds")
```

for Loop

A "for" loop loops explicitly over data (strings, lists, etc)

```
item_list = [1,2,3,4]
```

for Loop

A "for" loop loops explicitly over data (strings, lists, etc)

```
item_list = [1,2,3,4]

for item in item_list:
    print(item)
```

for Loop

A "for" loop loops explicitly over data (strings, lists, etc)

```
item_list = [1,2,3,4]
```

```
for item in item_list:  
    print(item)
```

```
# => prints in order:  1, 2, 3 and 4
```

for Loop

A "for" loop loops explicitly over data (strings, lists, etc)

```
item_list = [1,2,3,4]

for item in item_list:
    print(item)

# => prints in order:  1, 2, 3 and 4

for number in item_list:
    print(number * 2, end="|")
```

for Loop

A "for" loop loops explicitly over data (strings, lists, etc)

```
item_list = [1,2,3,4]

for item in item_list:
    print(item)

# => prints in order:  1, 2, 3 and 4

for number in item_list:
    print(number * 2, end="|")

# => 2|4|6|8
```

Range

You can iterate over a sequence of numbers with "range"

```
range(3)  
# => 0, 1, 2
```

Range

You can iterate over a sequence of numbers with "range"

```
range(3)
```

```
# => 0, 1, 2
```

```
range(5, 10)
```

```
# => 5, 6, 7, 8, 9
```

Range

You can iterate over a sequence of numbers with "range"

```
range(3)
# => 0, 1, 2

range(5, 10)
# => 5, 6, 7, 8, 9

range(2, 12, 3)           # Iterate with 3 steps between each number
# => 2, 5, 8, 11
```

Range

You can iterate over a sequence of numbers with "range"

```
range(3)
# => 0, 1, 2

range(5, 10)
# => 5, 6, 7, 8, 9

range(2, 12, 3)           # Iterate with 3 steps between each number
# => 2, 5, 8, 11

for n in range(1,5):
    print(n)              # => prints in order: 1, 2, 3 and 4
```

while Loop

A "while" loop loops until condition is met

```
# Print all number up to 10000  
n = 1
```

while Loop

A "while" loop loops until condition is met

```
# Print all number up to 10000
n = 1

while n < 10000:
    print(n)
    n += 1                                # += is a shortcut for an addition
```

break and continue

break breaks out of the smallest enclosing loop

```
for n in range(2, 10):  
    if n == 6:  
        break  
    print(n, end=', ')  
# => 2, 3, 4, 5,
```

break and continue

break breaks out of the smallest enclosing loop

```
for n in range(2, 10):  
    if n == 6:  
        break  
    print(n, end=', ')  
# => 2, 3, 4, 5,
```

continue continues with the next iteration of the loop

```
for letter in "STELLAR":  
    if letter in "LE":  
        continue  
    print(letter, end='*')  
# => S*T*A*R*
```

Exercise Time !

1. Open the command line and type "Python"
2. Use the "input" function to ask the user (you) for a word
3. Using a loop, reverse this word (initiate an empty string first !)
4. Check if this word is a palindrome (spelled the same backwards and forwards)
5. Make your loop ignore the number "e"
6. Save the result in a file named "toto.txt"

- 1 Before we start...
- 2 Introduction
- 3 The Basics of Python
- 4 Control Flow in Python
- 5 Objects and Types**

Objects in Python

Everything (i) is an object and (ii) has an identity

```
isinstance(4, object)      # => True
isinstance("Hello", object) # => True
isinstance(None, object)   # => True
isinstance([1,2,3], object) # => True
isinstance(str, object)    # => True
```

- "Identity" is unique and fixed during an object's lifetime
- Objects contain pointers to their data blob
- Even small things take up a lot of space !

Objects in Python

Everything (i) is an object and (ii) has an identity

```
isinstance(4, object)      # => True
isinstance("Hello", object) # => True
isinstance(None, object)   # => True
isinstance([1,2,3], object) # => True
isinstance(str, object)    # => True

# The id function returns an object's "identity"
id(41)                      # => 4361704848 (e.g.)
```

- "Identity" is unique and fixed during an object's lifetime
- Objects contain pointers to their data blob
- Even small things take up a lot of space !

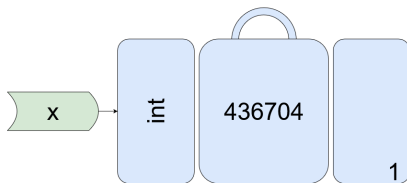
An analogy



Imagine that each Python object is a suitcase

- Each suitcase store a different value

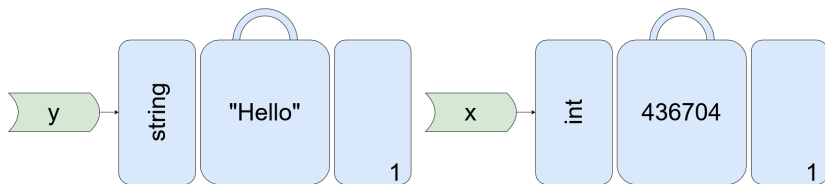
An analogy



Imagine that each Python object is a suitcase

- Each suitcase store a different value
- Variables are references to objects (the tags of our suitcases)

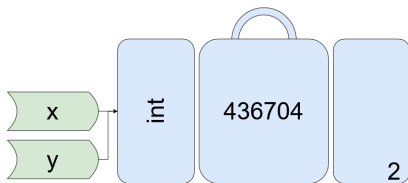
An analogy



Imagine that each Python object is a suitcase

- Each suitcase store a different value
- Variables are references to objects (the tags of our suitcases)

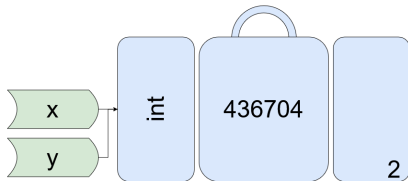
An analogy



Imagine that each Python object is a suitcase

- Each suitcase store a different value
- Variables are references to objects (the tags of our suitcases)
- Assigning from a variable ($x = y$) does not copy an object

An analogy



Imagine that each Python object is a suitcase

- Each suitcase store a different value
- Variables are references to objects (the tags of our suitcases)
- Assigning from a variable ($x = y$) does not copy an object
- Instead, it adds another reference to the same object

Exercise Time !

Type the following code and note the outputs

```
x = "hello world"
y = "hello "
y += "world"

id(x)
id(y)

x == y
x is y

4 > 3 is not True
```

Exercise Time !

Type the following code and note the outputs

```
x = "hello world"
y = "hello "
y += "world"

id(x)
id(y)

x == y
x is y

4 > 3 is not True
```

Why do "==" and "is" produce different outputs ?

is and ==

"is" checks if the suitcases are the same

"==" checks if they have the same stuff inside

is and ==

"is" checks if the suitcases are the same

"==" checks if they have the same stuff inside

You will *almost* always want to use "=="

Doggy bag

Python Basics

- Variables and types
- Operators
- Console input/output
- File manipulation

Control Flow

- Statements
- Loops

